* Number System Basics -:

e.g.

734 -> 700 + 30 + 4

$= 7\times10^2 + 3\times10^1 + 4\times10^0$

Digits => 0 to 9   10 digit

Base power 10

other number system -:

→ Binary 2

→ Octal - 8

→ Hexadecimal - 16

Binary → 0 and 1 digit

Binary → 2 1 power

e.g. $(1\ 0\ 1\ 0\ 0) = 1\times2^4 + 0\times2^3 + 1\times10^2$

$+ 0\times2^1 + 0\times2^0$

$= 16 + 4$

$= 20$

* Decimal to Binary :-

1. Repeatedly divide the number by 2, till you get 0.
2. Note down the remainders
3. Take the remainders in reverse.

2	34	
2	18	1
2	9	0
2	4	1
2	2	0
2	1	0
2	0	1

$(100101)_2$

* Adding 2 binary Numbers :-

	1	1				
	1	0	1	1	0	22
+	0	0	1	1	1	7
	1	1	1	0	1	29

s = 1

d = 1%2

c = 1/2

* Why Binary-:

We humans use a decimal or base 10 numbering system, presumably because people have 10 digit.

Early Computers were disigned around the decimal numbering system. This approch made the creation of computer logic capabilities unnecessarily complex and did not make efficient use of resources. (for example, 10 vacuum tubes were needed to represent one decimal digit).

To deal with the basic electronic states of ON and off, von Neumann suggested using the binary system.

ON		off
1	and	0

* Bitwise operators-:

AND	OR	NOT	XOR	left shift	right shift
&	|	~	^	<<	>>

Truth table-:

a	b	a & b	a | b	a ^ b	~a	~b
0	0	0	0	0	1	1
0	1	0	1	1	1	0
1	0	0	1	1	0	1
1	1	1	1	0	0	0

↑ Addition without carry.

* Properties of Bitwise Operators-:

a & 1 = if (a & 1) == 1
n is odd
else
n is even.

1) a & 0 = 0
2) 0 & a = a

3) $a \mid 0 = a$

$\boxed{\begin{array}{l} 4)\quad 0 \wedge 0 = a \\ 5)\quad a \wedge a = 0 \end{array}}$

6) $a \wedge 1 =$ → even+1

→ odd-1

7) $a \& 1 =$ → even+1

→ odd same

$$\begin{array}{r} 1\,0\,1\,1 \\ \mid \; 0\,0\,0\,1 \\ \hline 1\,0\,1\,1 \end{array}$$

8) $a \& b = b \& a$

9) $a \mid b = b \mid a$ } Commutative property

10) $a \wedge b = b \wedge a$

Associate property

$a \& b \& c = c \& b \& a$

Example -:

$$a \wedge b \wedge a \wedge d \wedge b$$
$$= a \wedge a \wedge b \wedge b \wedge d$$
$$= 0 \wedge 0 \wedge d$$
$$= d$$

* Single Number :

arr[5] = 6 9 6 10 9

$\hookrightarrow$ 10

Idea :

Take XOR of all elements

TC: O(N)

SC: O(1)

* Left shift operator : (<<)

8 bits	7	6	5	4	3	2	1	0	
a=10	0	0	0	0	1	0	1	0	= 10
a<<1	0	0	0	1	0	1	0	0	= 20

a<<2 = 40 = 10×2^2

largest 8 bit number 255

Important Result :

$1 << N = 1 \times 2^N$

$\boxed{= 2^N \longleftarrow O(1) \text{ time}}$

* Right shift operator :- (>>)

$a = 10$

$$0\ 0\ 0\ 0\ 1\ 0\ 1\ 0 = 10$$

$$a >> 1 \quad 0\ 0\ 0\ 0\ 0\ 1\ 0\ 1 = 10/2$$

$$a >> 2 \quad = \frac{10}{4}$$

Generalisation :-

$$\boxed{a >> i = \frac{a}{2^i}}$$